胡傲东

📱 18085741680 ✉️hu.aodong@icloud.com GitHub 个人主页

核心竞争力

客户端工程师，面向桌面端复杂交互与长任务工作流，擅长在 Qt/QML 与 Tauri 场景下处理渲染流畅度、线程解耦、资源占用与跨平台交付问题。能够把多阶段推理或系统能力封装成可恢复、可追踪、可操作的客户端体验。

Qt 6 / QML · Tauri · FFmpeg · 跨平台桌面 · 多线程调度 · 实时预览

教育背景

华中科技大学 · 集成电路设计与集成系统（本科） 2022.09 - 2026.06

专业技能

语言与框架
C++17、Qt 6 / QML、TypeScript、React 18、Tauri、FFmpeg

交互与架构
MVVM、组件拆分、异步状态流、任务状态机、增量刷新、错误回滚

平台能力
Windows / macOS / Linux 跨平台、桌面端打包分发、IPC、文件系统与媒体处理

性能治理
QThread、Signal-Slot、帧缓冲复用、长任务恢复、内存占用控制、卡顿排查

工程工具
CMake、Vite、Git、CI/CD、日志与性能分析

项目经历

端到端 AI 视频生成系统（跨平台多模态任务平台） ↗ 全栈

字节剪映 · 工程营实习生小组长（客户端/多模态） · 2025.11 - 至今

项目定位：将跨平台桌面端从“提交任务后等待结果”的弱交互工具重构为支持实时分镜编辑、进度反馈、断点恢复与视频导出的 AI 生成工作台，在 2 周内完成 MVP 并将预览帧率从 30fps 提升至 60fps
架构取舍：直接在 UI 线程轮询推理状态与刷新预览会导致长任务场景下界面冻结；因此采用 Qt/QML MVVM 分层、QThread + Signal-Slot 解耦交互线程与计算线程，并将任务状态、预览帧和导出链路拆成可独立刷新的异步模块
机制设计：统一创建/暂停/恢复/取消/重试的任务协议，前端仅消费可追踪的状态流与增量结果，避免页面级状态纠缠；结合帧缓冲复用与异步回填，让多模态实时预览在长任务下仍保持可操作性
协同优化：客户端侧按资源压力触发任务降级与后端限流协作，减少显存争用导致的卡顿与失败，长任务恢复率 95%+，内存峰值降低 30%，缓存复用率从 25% 提升至 65%
技术栈：Qt/QML/C++ · Go/Gin · FastAPI · Redis/Asynq · Docker · FFmpeg · Qwen / Stable Diffusion / Stable Video Diffusion / Text-to-Speech

AI 智能教学平台 ↗ 全栈

本科毕业设计 · 全栈开发（侧重跨平台客户端） · 2025.11 - 至今

项目定位：将教学系统客户端重构为覆盖 Windows/macOS/Linux 桌面端、企业微信 H5 与移动端的统一交互层，使学生画像、学习轨迹与 AI 问答能够跨端连续使用
架构取舍：未继续采用课程中心驱动的静态页面流，因为该模式难以承载跨课程学情追踪与端侧智能交互；转而以学生画像和学习档案为核心抽象前后端协议，并让桌面端与 Web 端共享同一套业务语义
机制设计：桌面端通过 Vite + React + Tauri 承载统一代码库分发，并接入 NPU 本地推理形成离线问答能力；客户端状态层同时协调本地推理、云端知识检索与结果回填，避免多端结果不一致
工程结果：平台在试点场景中稳定支持 30+ 并发用户，AI 答疑准确率达到 85%+，且 Docker Compose 一键部署显著降低了跨端联调与试点交付成本
技术栈：Vite/React/TypeScript/Tauri · Go/Gin/GORM · Python/FastAPI · MySQL/MinIO · Docker Compose · Qwen3 · NPU acceleration

macOS 系统优化工具（SwiftSweep） ↗ Apple

个人独立项目 / 开源工具 · 2024.10 - 2024.12

项目定位：为解决 macOS 用户存储空间焦虑与常见清理工具高授权风险，基于 Swift 5.9 与 SwiftUI 独立开发集系统深度清理、相似文件检测与特权隔离于一体的原生维护工具（GUI + CLI），实现了 14+ 核心系统维护功能并成功完成 Apple 官方公证发布
架构与安全设计：摒弃传统清理软件索要全局 root 权限的危险模式，采用 XPC Helper + SMAppService 构建进程间通信隔离层，主应用在沙盒与受限权限下运行，彻底阻断了提权漏洞攻击面；设计了 dry-run 预演、基线白名单与严格的 symlink（符号链接）逃逸检测三重防护体系，实现 100% 零误删系统核心进程数据
算法与性能调优：面对数以万计的相似图片与视频检测导致的内存暴涨与 CPU 满载问题，引入基于 pHash（感知哈希）与 LSH（局部敏感哈希）降维检测算法；利用 Swift Task / Async 机制实现流式处理与内存回收，将重复媒体文件扫描速度提高了 3 倍，内存峰值控制在 200MB 以内
技术栈：Swift 5.9 / SwiftUI · XPC 进程间协议通信 · SMAppService · pHash / LSH 特征匹配算法 · Swift 包管理器 (SPM) · Developer ID 证书体系与 Notarization